Yucheng Jin (Yucheng 913170112253)

Problem 1.

(1) The Scenario is more suitable for cloud computing. Reasons:
① The camera needs to be in the wild for several days using battery power, edge computing requires high energy consumption at end device so it is not suitable in terms of energy.
② There is no need for real-time processing, but the system should report the appearance of the wildlife. so cloud computing is preferred because it requires data to be uploaded to the cloud and send report of successful detection to other devices.
③ Recognizing wildlife requires high computation ability, and cloud computing can meet high computation requirements.

(2) ① Function Node: specifies the information needed, such as battery level and GPS information, by editing message headers (msg.headers) and message payload (msg.payload).

② HTTP Request Node: sends the message to the remote server by the POST method and receives the response.

③ Debug Node: prints out the response in the debugging sidebar.

(3) Vanishing gradient problem: the gradient tends to get smaller as we move backward through the hidden layers, which means that neurons in the earlier layers learn much more slowly than neurons in later layers. The opposite phenomeon may also occur, which is exploding gradient problem.

(4)

	original time		Quantization time	
Layer 1	$\frac{128\times32\times3\times3\times12\times12}{10^9}$	$= 5.308\times10^{-3}$s	$5.308\times10^{-3}\div 45$	$= 3.981\times10^{-5}$s
		1.592×10^{-2}s	$1.592\times10^{-2}\div 45$	
Layer 2	$\frac{256\times192\times3\times3\times6\times6}{10^9}$	$= 0.1592$s	$0.1592\div 45 = 3.981\times10^{-3}$s	
Layer 3	$\frac{256\times256\times6\times6}{10^9}$	$= 2.359\times10^{-3}$s	$2.359\times10^{-3}\div 45$	
Layer 4	$\frac{256\times256}{10^9}$	$= 6.554\times10^{-5}$s	$6.554\times10^{-5}\div 45$	

Layer 2 should be quantized.
Times in red boxes.

Scanned with CamScanner

Problem 2.

(1) channel 0 output: channel 1 output:

11	9	9
13	12	11
12	10	10

7	8	9
9	8	10
5	7	6

(2) $dE/dW =$

$dE/dX =$

(3) Like what we do in HW2:

conv1 = $3 \times 28 \times 28 \times 28 \times 5 \times 5 \times 2 = 3,292,800$

conv2 = $28 \times 32 \times 8 \times 8 \times 7 \times 7 \times 2 = 5,619,712$

fc1 = $32 \times 4 \times 4 \times 100 \times 2 = 102,400$

fc2 = $100 \times 10 \times 2 = 2,000$

total = 9,016,912

(4) $28 \times 0.6 = 16.8$, so 17 (or 16) channels of the first Conv layer.
(output)

input image size (3, 32, 32)

```
CNN = keras.Sequential([
    keras.layers.Conv2D(filters=17, kernel_size=(5,5), strides=1,
                        padding='valid', activation='relu', input_size=(32,32,3))

    keras.layers.MaxPooling(kernel_size=(2,2), strides=2)
    keras.layers.Conv2D(filters=32, kernel_size=(7,7), strides=1,
                        padding='valid', activation='relu', input_size=(14,14,17))

    keras.layers.MaxPooling(kernel_size=(2,2), strides=2)
    Keras.Dense(units=100, activation='relu')
    keras.Dense(units=10)
```

])

Scanned with CamScanner

Problem 3.

(1) ① 32 ② blockIdx.x * blockDim.x + threadIdx.x

③ blockIdx.y * blockDim.y + threadIdx.y

④ global_row * numAcolumns + cur_col_of_A

⑤ cur_col_of_A * numBcolumns + global_col

⑥ global_row * numCcolumns + global_col

(2) ① 16 ② __shared__ float subTileA[TILE_WIDTH][TILE_WIDTH]
__shared__ float subTileB[TILE_WIDTH][TILE_WIDTH]

③ global_row * numAColumns + m * TILE_WIDTH + threadx

④ (m * TILE_WIDTH + thready) * numBcolumns + global_col

⑤ thready ⑥ k ⑦ k ⑧ threadX ⑨ global_row * numCcolumns + global_col

(3) $Fraction_{parallel} = 85\%$, $Speedup_{parallel} = 8 \times 2048$.

$$Ideal\ Speedup = \frac{1}{(1-0.85) + \frac{0.85}{8\times2048}} = 6.664.$$

(4) For TILE_WIDTH=16, each thread block uses $2 \times 256 \times 4B = 2KB$ of shared memory. # Threads = 2048 # Blocks = 8.

$Fraction_{parallel} = 1$, $Speedup_{parallel} = 8 \times 2048$,

$$Ideal\ speedup = \frac{1}{(1-1) + \frac{1}{8\times2048}} = 16,384.$$

We could change TILE_WIDTH to further improve computation.

Scanned with CamScanner

Problem 4.

(1) Since $\frac{R_i}{R_j} = \frac{\sqrt{L_i}}{\sqrt{L_j}}$, let $\sqrt{L_1} = \sqrt{64} = 8$ $\sqrt{L_2} = \sqrt{49} = 7$ $\sqrt{L_3} = \sqrt{25} = 5$

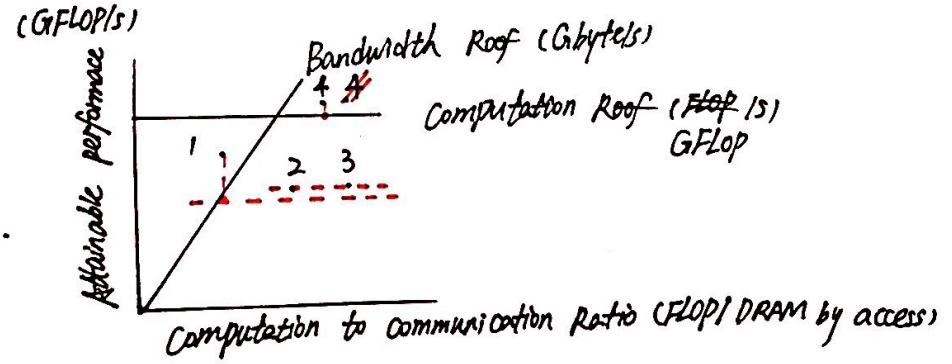
$\therefore R_1 = \frac{\sqrt{L_1}}{\sqrt{L_1}+\sqrt{L_2}+\sqrt{L_3}} \times 100\% = 40\%$ => L_1's resources

$R_2 = \frac{\sqrt{L_2}}{\sqrt{L_1}+\sqrt{L_2}+\sqrt{L_3}} \times 100\% = 35\%$ => L_2's resources

$R_3 = \frac{\sqrt{L_3}}{\sqrt{L_1}+\sqrt{L_2}+\sqrt{L_3}} \times 100\% = 25\%$ => L_3's resources

(2)

a.



b. $4 > \frac{2}{3} = 3 > 1$

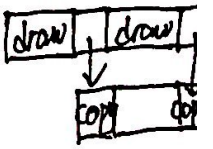
c. 3, because compared to 2. it has the same attainable performace but a higher computation to communication ratio.

(3) Because BNN encodes +1, -1 as 0, 1,

a.

b_1	b_2	$b_1 \times b_2$		b_1	b_2	b_1 XOR b_2
+1	+1	+1	=>	0	0	0
+1	-1	-1		0	1	1
-1	+1	-1		1	0	1
-1	-1	+1		1	1	0

b. Double buffering uses two buffers to hold a block of data. first, one buffer is drawn, then the data will be copied to the other buffer and it is drawn, and readers could see a complete version of data.



(4) ① Resource utilization: FPGA is good at resource management, by its REsource ALlocation Management (REALM). it can analyze resource allocation among the layers and determine the most efficient allocation to minimize total network latency. REALM: $\frac{R_i}{R_j} = \frac{\sqrt{C_i}}{\sqrt{C_j}}$

② Computation Latency: FPGA reduces the delay in processing data, by a "Cache" architecture and a special pipeline architecture.

Scanned with CamScanner